

FORMAL LANGUAGES AND COMPILERS

prof.s Giovanni Agosta, Luca Breveglieri and Daniele Cattaneo

Exam of MONDAY 19 JANUARY 2026 – Laboratory Part

WITH SOLUTIONS

LAST NAME (SURNAME) + FIRST NAME:

(capital letters please)

MATRICOLA:

(or PERSON CODE)

SIGNATURE:

TEACHER: ☐ Prof. G. AGOSTA – ☐ Prof. L. BREVEGLIERI – ☐ Prof. D. CATTANEO

INSTRUCTIONS – READ CAREFULLY:

- The exam is open book: textbooks and personal notes are permitted.
- Pencil writing is permitted, except on the cover page (this sheet).
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets and do not replace the existing ones.
- Time: lab part 1 h – theory part 2 h.

SCORING AND GRADE ATTRIBUTION:

- The exam is in written form and consists of two parts:

Theory syntax and semantics of languages, divided in four sections: (75% of the final grade)

- 1) regular expressions and finite automata (1 exercise × 10 points)
- 2) syntax analysis and parsing methodologies (1 exercise × 10 points)
- 3) grammar design – translation – semantic analysis (2 exercises × 5 points)
- 4) *bonus* question (1 exercise × 5 points)

Lab compiler design by Flex and Bison (25% of the final grade)

- 1) basic questions (30 points)
- 2) *bonus* question (5 points)

- For the theory part to be valid, the candidate must achieve a grade of at least 5/10 in each of the three sections 1, 2 and 3. If this is not the case, section 4 is not evaluated. Section 4 has no threshold on its own. Correctly answering all the questions in sections 1, 2 and 3 allows the candidate to achieve a grade of 30/30 (but not *cum laude*) for the theory part.
- For the lab part to be valid, the candidate must achieve a grade of at least 15/30 in the basic questions. The bonus question is evaluated only if the threshold is reached.
- To pass the exam, the candidate must succeed in both parts (theory and lab), in one call or more calls separately, but within one year (12 months) between the two parts.
- The final grade is the weighted average of the theory part (75 %) and of the lab part (25 %), and must be $\geq 18/30$ (*cum laude* $\geq 32/30$).

1 Basic Questions (30 points)

1. Consider the implementation of the ACSE compiler shown in class.

Modify the specification of the lexical analyser (`flex` input) and the syntactic analyser (`bison` input) and any other source file required to extend the LANCE language with the ability to assign a **list of inline expressions** to multiple **array** elements with a single statement.

This new statement appears like this:

$$\langle array \rangle = \{ \langle exp. 1 \rangle, \langle exp. 2 \rangle, \dots \langle exp. n \rangle \}$$

When the statement is encountered, the compiler generates code that **assigns each expression** within the curly brackets **to each of the elements** within $\langle array \rangle$, starting with the first one (i.e. at index zero) and then proceeding in increasing order.

For example, assuming that a is an array with 3 elements, the following line of code assigns 7 to $a[0]$, 2 to $a[1]$ and 5 to $a[2]$:

```
a = {7, 2, 5};
```

The list of elements must have **at least one element**, and it may contain fewer elements than the length of the array — in that case, the remaining array items are left **unmodified**. If more elements than the length of the array are present, the remaining elements are ignored, and during compilation a **warning** is printed notifying the users of their mistake.

As previously stated, the list items can be any valid expression in the language. When an expression in the list references the same array that we are assigning, it assumes all previous expressions in the list to have already been assigned. For example, the following line of code first assigns 8 to $a[0]$, then assigns 15 to $a[1]$ from the computation $a[0] + 7 = 8 + 7 = 15$. Element $a[2]$ is left unmodified.

```
a = {8, a[0]+7};
```

The following program provides an additional example of the behavior of the statement.

```
int a, b[3];

a = 3;
b[2] = 2;
b = {0, 1};
b = {b[1]+a, b[2]+a*2, b[0]+a*3, b[0]};

// b is now {4, 8, 13}
```

- (a) Define the tokens (and the related declarations in **scanner.l** and **parser.y**). (1 points)
- (b) Define the syntactic rules or the modifications required to the existing ones. (3 points)
- (c) Define the semantic actions needed to implement the required functionality. (21 points)

Solution

The solution is shown below in *diff* format. All lines that begin with '+' were added, while the lines that begin with '-' were removed. **It is not required and it is not encouraged to provide a solution in *diff* format to get the maximum grade.**

```

diff --git a/acse/parser.y b/acse/parser.y
--- a/acse/parser.y
+++ b/acse/parser.y
@@ -24,6 +24,9 @@ void yyerror(const char *msg)
    emitError(curFileLoc, "%s", msg);
}

+// The array which is being assigned to by the current array assign statement
+t_symbol *curArray;
+
+
+/*
@@ -88,6 +91,7 @@ void yyerror(const char *msg)

    %type <var> var_id
    %type <reg> exp
+    %type <integer> array_assign_list

+/*
+ * Operator precedence and associativity
@@ -196,6 +200,52 @@ assign_statement
{
    genStoreRegisterToArrayElement(program, $1, $3, $6);
}
+ | var_id ASSIGN LBRACE
+ {
+    // Check that the destination of the assignment is in fact an array
+    if (!isArray($1)) {
+        yyerror("array assignment to variables is illegal");
+        YYERROR;
+    }
+    // Copy the symbol object of the array to a global so that it can be
+    // accessed by the semantic actions in array_assign_items.
+    curArray = $1;
+ }
+ array_assign_list RBRACE
+;
+
+array_assign_list
+ : array_assign_list COMMA exp
+ {
+    // We use the semantic value of array_assign_items to store the number of
+    // elements in the assignment list found so far.
+    // Only generate code if the element we will assign next is within bounds.
+    if ($1 < curArray->arraySize) {
+        // The element is within bounds, generate code to load the index into a
+        // register as required by genStoreRegisterToArrayElement
+        t_regID rIdx = getNewRegister(program);
+        genLI(program, rIdx, $1);
+        // Generate the store to the array element
+        genStoreRegisterToArrayElement(program, curArray, rIdx, $3);
+    } else if ($1 == curArray->arraySize) {
+        // The list of expressions is too large, print a warning but only for
+        // the first out of bounds element.
+        printf("warning: too many elements in array assignment, extras ignored\n");
+    }
+    // There is one more one element than before
+    $$ = $1 + 1;
+ }
+ | exp
+ {
+    // The action for the base case of the recursion is the same as the

```

```

+ // recursive case, but we are always assigning to the first element (which
+ // is always in range because zero-length arrays are illegal in LANCE).
+ t_regID rIdx = getNewRegister(program);
+ genLI(program, rIdx, 0);
+ genStoreRegisterToArrayElement(program, curArray, rIdx, $1);
+ // There is only one element so far
+ $$ = 1;
+ }
+
+
+ /* An if statements first computes the expression, then jumps to the 'else' part
diff --git a/tests/array_assign/array_assign.src b/tests/array_assign/array_assign.src
--- /dev/null
+++ b/tests/array_assign/array_assign.src
@@ -0,0 +1,13 @@
+int a, i;
+int b[3];
+
+read(a);
+b[2] = 2;
+b = {0, 1};
+b = {b[1]+a, b[2]+a*2, b[0]+a*3, b[0]};
+
+i = 0;
+while (i < 3) {
+ write(b[i]);
+ i = i + 1;
+}

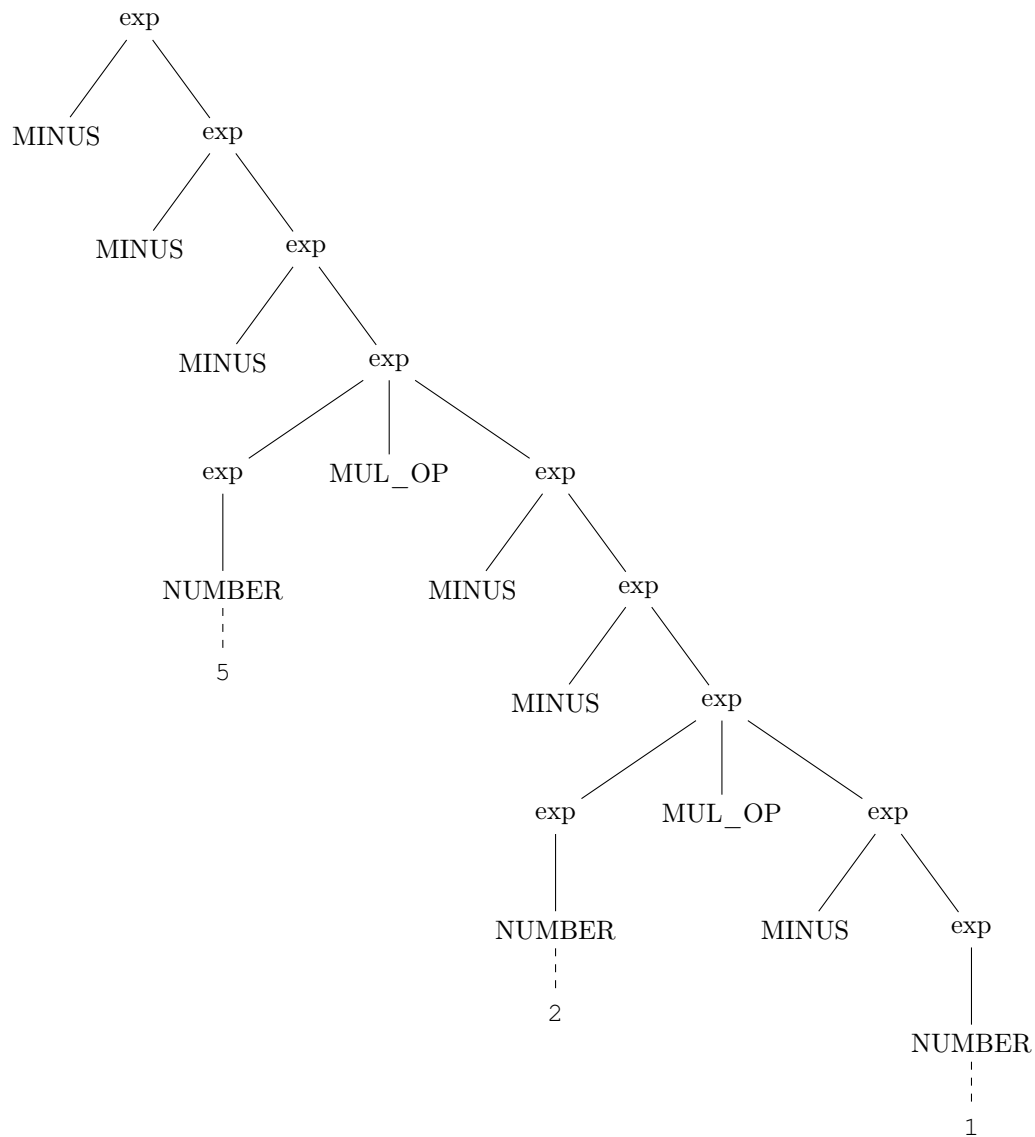
```

2. Given the following LANCE code snippet:

$$---5*--2*-1$$

write down the syntactic tree generated during the parsing with the Bison grammar described in `parser.y` *starting from the `exp` nonterminal*. (5 points)

Solution



2 Bonus Question

- Briefly explain in plain English words how to modify the answer given to the questions about modifications to ACSE in order to allow the programmer to specify a starting element from which the items are assigned in the array. For example the following program should result in array *a* containing the values {1,2,3}:

```
int a, b[3];
a = 6;
b = {1};
b[7-a..] = {2, 3}; // '7-a' is an expression
```

Is it still always possible to warn the user when too many elements are present in the list? (2 points)

- Consider the following snippets of ACSE compiler code, which define a new statement **vil**:

scanner.l	
"vil"	{ return VIL; }

parser.y	
statement	: /* ... */ vil SEMI ;
vil	: exp VIL exp { if (\$1 + 1 == \$3) { printf("warning: oh no! %d is one more than %d\n", \$3, \$1); return 0; } } ;

And the following LANCE program that uses the vil statement defined as above:

vil.src	
3 vil 4; hello	

Can you tell if the compiler prints the message programmed into the semantic action for statement vil, and which numbers are printed within it? In general, what would happen afterwards if the message is printed? Does the compiler produce additional errors? (Hint: for the second part of the question, you can look at the implementation of the function *parseProgram()* found in *parser.y*.) (3 points)

Solution

1. The grammar for the statement shall be changed to the following:

parser.y
<pre> assign_statement : /* ... */ var_id ASSIGN LBRACE array_assign_list RBRACE var_id LSQUARE exp DOTS RSQUARE ASSIGN LBRACE array_assign_offset_list RBRACE ; </pre>

where DOTS is defined in *scanner.l*:

scanner.l
<pre> ".." { return DOTS; } </pre>

Like for the version of the statement without starting index, the only semantic action outside of the list appears before it, and the only addition is that it stores the semantic value of the starting index expression (a register ID) into a global variable. This global variable is used within *array_assign_offset_list* to compute the indices. Since now the indices are not constant, and depend on the value of the starting index expression — which is not known at compile time — the semantic actions in *array_assign_offset_list* cannot detect any array out of bounds error.

2. The message is in fact printed, not because $3 + 1 = 4$, but rather because the evaluation of the two expressions results in two back-to-back calls to *getNewRegister()* to obtain the semantic values of the two *exp* nonterminals. So, in the semantic function for *vil*, the two values \$1 and \$2 do in fact satisfy the expression. The program only contains one line, and as a result we can easily predict that $\$1 = 1$ and $\$2 = 2$. Therefore the warning is printed and will be:

“warning: oh no! 2 is one more than 1”

After the warning is printed, the *return* statement executes, which causes the *yyparse()* function to return immediately. The return value of *yyparse()* is ignored, and the error message counter has not been modified, so the *parseProgram()* function exits normally. Therefore, compilation continues ignoring the rest of the file, producing no additional errors despite the following line being syntactically invalid.